
Курсовая работа

Визуализация течения жидкости с использованием WebGL

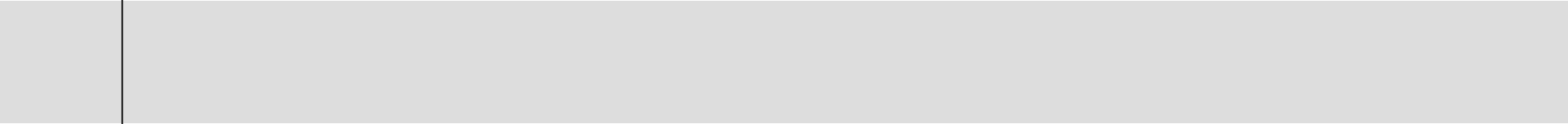
Студент	Федуков А. А. (ИУ9-52Б)
Руководитель	Царев А. С.

Введение

Визуализация течения жидкости —
вычислительно сложная задача

- Выберем алгоритм визуализации
- Реализуем при помощи WebGPU

Цель: реализовать интерактивную 2D симуляцию
жидкости в веб-приложении



01

Изучение предметной области

Подходы к симуляции жидкости

По подходу

- Эйлера (сетка)
- Лагранжевы (частицы)
- Гибридные

По цели

- Физически обоснованные
 - Эвристические
-

02

Выбор алгоритма визуализации жидкости

Алгоритмы визуализации

Рассмотрены

- Stable Fluids, SPH, Shallow Water, FLIP, MLS-MPM

Выбран

- Stable Fluids (эйлеров, сетка)
 - Упрощенный Навье–Стокс
 - Сетка хорошо ложится на WebGPU
-

03

Исследование алгоритма и возможностей WebGPU

Уравнения Навье – Стокса

$$\begin{cases} \frac{\partial \mathbf{u}}{\partial t} = -(\mathbf{u} \cdot \nabla) \mathbf{u} - \frac{1}{\rho} \nabla p + \nu \nabla^2 \mathbf{u} + \mathbf{f}, \\ \nabla \cdot \mathbf{u} = 0. \end{cases}$$

Stable Fluids

$$\frac{\partial \mathbf{u}}{\partial t} = P \left(-(\mathbf{u} \cdot \nabla) \mathbf{u} + \nu \nabla^2 \mathbf{u} + \mathbf{f} \right) .$$

$$\mathbf{w}_0(\mathbf{x}) \rightarrow \mathbf{w}_1(\mathbf{x}) \rightarrow \mathbf{w}_2(\mathbf{x}) \rightarrow \mathbf{w}_3(\mathbf{x}) \rightarrow \mathbf{w}_4(\mathbf{x})$$

$$\mathbf{w}_1(\mathbf{x}) = \mathbf{w}_0(\mathbf{x}) + \Delta t \mathbf{f}(\mathbf{x}),$$

$$\mathbf{w}_2(\mathbf{x}) = \text{Advect}(\mathbf{w}_1, \Delta t),$$

$$\mathbf{w}_3(\mathbf{x}) = \text{Diffuse}(\mathbf{w}_2, \nu, \Delta t),$$

$$\mathbf{w}_4(\mathbf{x}) = \mathcal{P}(\mathbf{w}_3(\mathbf{x})),$$

https://pages.cs.wisc.edu/~chaol/data/cs777/stam-stable_fluids.pdf

WebGPU

- Современная технология
 - Пока еще в тестовом режиме
 - Низкоуровневый доступ к GPU
 - Простой API, чем-то похожий на Vulkan
 - Шейдеры на WGSL (похож на Rust)
-

04

Реализация алгоритма

Параметры симуляции

- Сетка:
 - 2D $N \times N$, где $N=250$,
 - область $L=750 \times 750$ px
 - шаг ячейки $\Delta x=3$
 - шаг времени $\Delta t=1/60$
 - Буферы (x2 для ping-pong):
 - скорость (u)
 - давление (p)
 - краска (dye)
 - дивергенция (для Пуассона)
 - Динамические параметры симуляции (uniform):
 - Вязкость $\nu=0.005$
 - Рассеивание $dissipation=0.999$
 - Завихренность $vorticity=25$
 - Гравитация $gravity=-20$
 - Добавление краски
 $faucet_dye=200$
 - Параметры мыши
 $mouse\ pos, vel$
-

Конвейер WGSL

- Этап 0. Общие модули (общие функции используемые во всех шейдерах)
 - 0) header.wgsl
 - Этап 1. Силы (формирование внешних воздействий на поле скорости)
 - 1) add_force.wgsl
 - 2) vortex_force.wgsl
 - 3) gravity.wgsl
 - Этап 2. Перенос и вязкость (перенос скорости и её сглаживание/диффузия)
 - 4) advect_vel.wgsl
 - 5) viscosity.wgsl
 - Этап 3. Проекция (обеспечение несжимаемости через давление)
 - 6) clear_pressure.wgsl
 - 7) divergence.wgsl
 - 8) pressure_jacobi.wgsl
 - 9) subtract_gradient.wgsl
 - Этап 4. Краситель (визуализация потока через добавление и перенос dye)
 - 10) add_dye.wgsl
 - 11) splat_dye.wgsl
 - 12) advect_dye.wgsl
 - Этап 5. Рендер
 - 13) render.wgsl
-

Шейдер решения давления

```
// Projection

// pressure.wgsl
// divergence + p_in -> p_out (one Jacobi iteration)

fn p_at(x: i32, y: i32) -> f32 {
    let N = i32(N_u32());
    let xi = clamp(x, 0, N - 1);
    let yi = clamp(y, 0, N - 1);
    return p_in[idx(u32(xi), u32(yi))];
}

@compute @workgroup_size(16,16,1)
fn cs_main(@builtin(global_invocation_id) gid: vec3<u32>) {
    let x = i32(gid.x);
    let y = i32(gid.y);
    if (x < 0 || y < 0 || x >= i32(N_u32()) || y >= i32(N_u32())) { return; }

    let i = idx(u32(x), u32(y));

    // pass-through vel/dye so ping-pong pressure iterations doesn't scramble them
    vel_out[i] = vel_in[i];
    dye_out[i] = dye_in[i];

    let pl = p_at(x - 1, y);
    let pr = p_at(x + 1, y);
    let pb = p_at(x, y - 1);
    let pt = p_at(x, y + 1);

    let div = divergence[i];

    p_out[i] = (pl + pr + pb + pt - div) * 0.25;
}
```

05

Разработка веб-приложения

Typescript + Next.js + Vercel

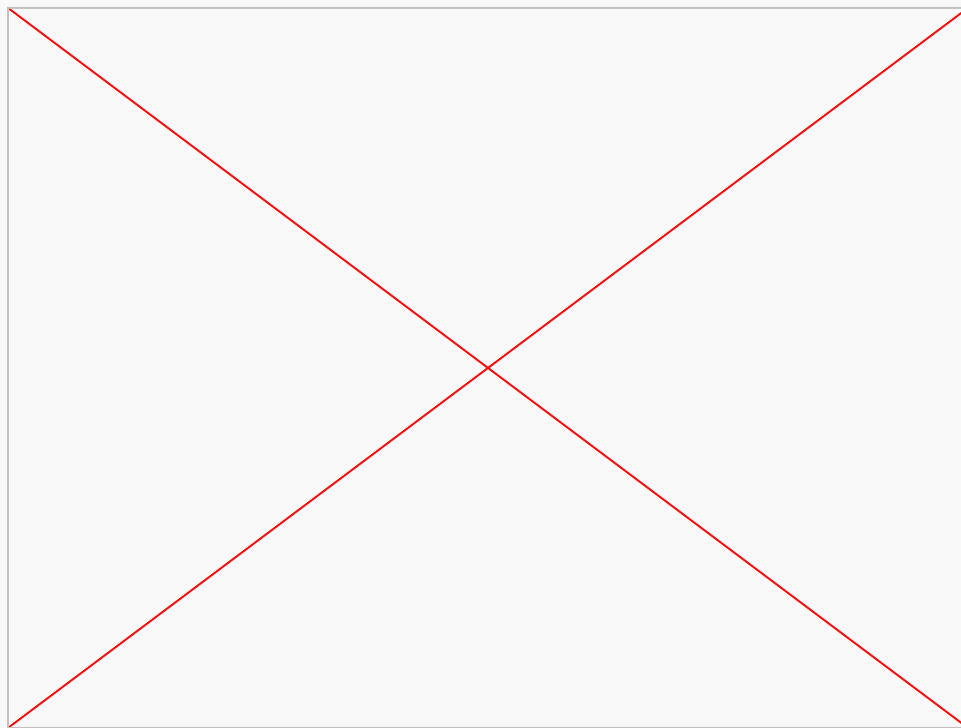
Цикл отрисовки

```
export function Render(  
  context: GPUCanvasContext,  
  device: GPUDevice,  
  pipelines: PipelinesResult,  
  bindGroups: BindGroupsResult,  
  uniformBuffers: InitUniformsResult,  
  params: WebGPUParams,  
  ping: number  
): number {  
  let p = ping  
  
  // 0) Update uniforms  
  writeBufferUniforms(device, uniformBuffers.uniformBuffer, params)  
  writeBufferMouse(device, uniformBuffers.mouseBuffer, params)  
  
  const encoder = device.createCommandEncoder()  
  
  const wx = Math.ceil(N_CELLS.x / WORKGROUP_SIZE)  
  const wy = Math.ceil(N_CELLS.y / WORKGROUP_SIZE)  
  
  const cpass = encoder.beginComputePass()  
  
  // -----  
  // 1) ADD FORCES (w0 -> w1)  
  // -----  
  // 12) add force to velocity  
  cpass.setPipeline(pipelines.addForcePipeline)  
  cpass.setBindGroup(0, bindGroups.uniformBG)  
  cpass.setBindGroup(1, bindGroups.simBG[p])  
  cpass.dispatchWorkgroups(wx, wy)  
  p ^= 1  
  
  // 13) add vortex force to velocity  
  cpass.setPipeline(pipelines.vortexForcePipeline)  
  cpass.setBindGroup(1, bindGroups.simBG[p])  
  cpass.dispatchWorkgroups(wx, wy)  
  p ^= 1
```

06

Тестирование,
оптимизация,
результат

Демонстрация



Заключение

**Реализован алгоритм Stable Fluids при
помощи WebGPU и Next.js**

<https://github.com/sorrtory-vercel/stable-flids-webgpu>
